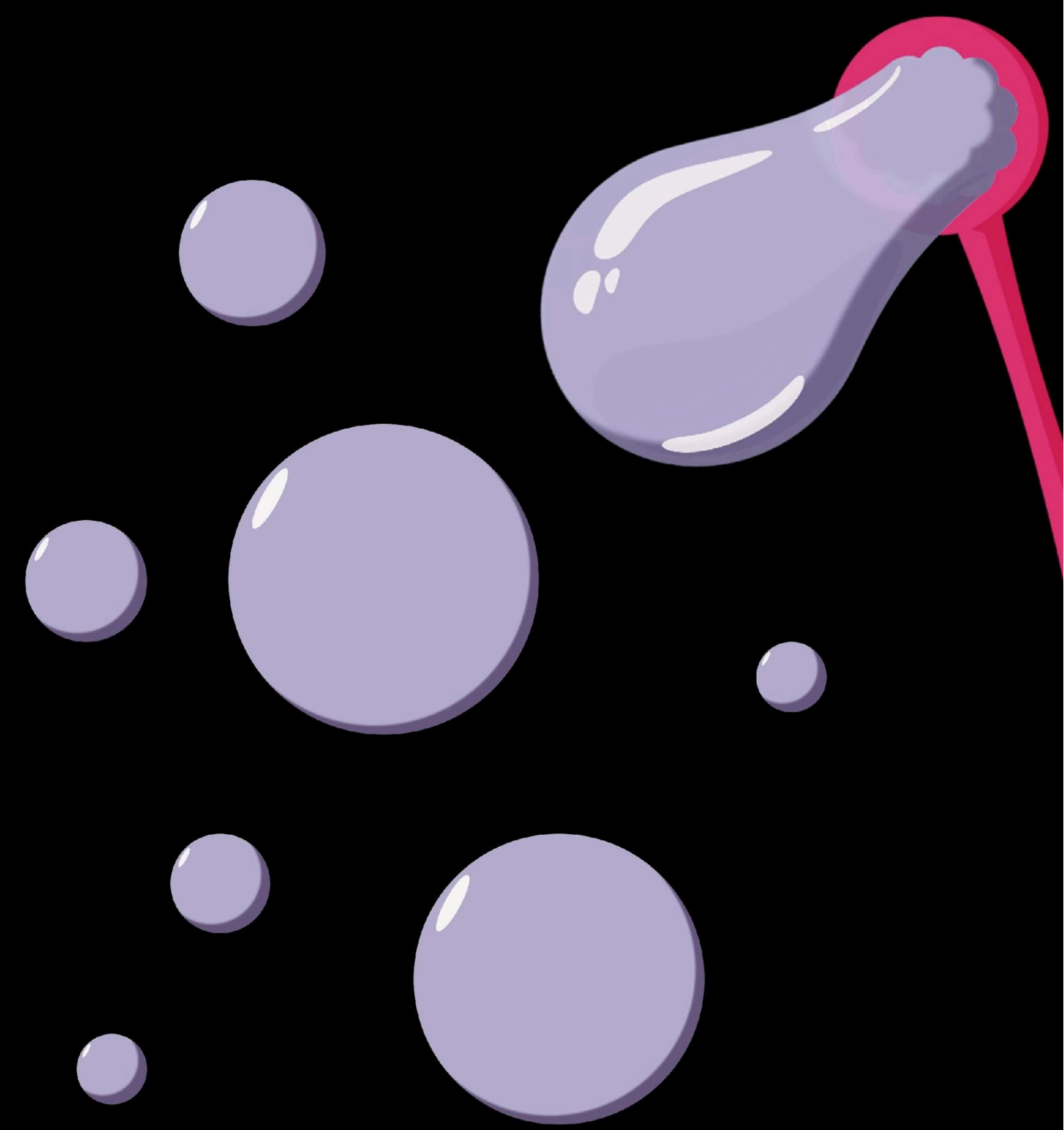

Bubblify

Bubblify (Next.js Edition)

A major catastrophe has just happened! All the key players in the bubble industry have been relying on the same manufacturer which just recently went through a really bad lawsuit which resulted in them going bankrupt! It is already May and the summer is almost here and kids want their bubbles! A new emerging company called **Bubblify** has decided to jump on this chance and become the new big player in the bubble industry. They want their business to go mainly through their website, which has not been made yet… That's where you come in! You are going to build a website for them before the summer season begins. Please, do not disappoint them nor all the soon-to-be bubbleless children…

TypeScript

All code written in this course should use TypeScript and this assignment is no exception. You should use create-next-app to scaffold the project.

Design

There is no Figma document, so you are free to style the application as you wish.

Resources

All packages are available.

External API

A **server** is provided and can be downloaded in the **Assignment** section in **Canvas**. This server should provide you with all the functionality you need, but if you want to change it, feel free. If you change the server code, include the new server code with your submission and update the **README.md** accordingly.

Getting started

1. Create a project using create-next-app (select TypeScript, Tailwind, App Router).
2. Extract the external API folder and run npm install inside it.
3. Start the API by running `node index.js`.
4. Verify it works: curl http://localhost:3500/api/bubbles.
5. Use `http://localhost:3500/docs` to see which methods are available in the external API.

Assignment description

All the functionality in the assignment is enlisted here below. You must utilize the **App Router** for all pages.

- **(10%)** Implement a site where you can look at all the available bubble products (e.g. **http://localhost:3000/bubbles**)
    - **Requirements**

        - This must be a **Server Component** that fetches data directly from the API.
        - All product images must be rendered using the Next.js <Image /> component to ensure proper optimization.
        - Products should be arranged in a grid consisting of 3 columns, and a consistent gap between each product as well.
        - The product list should display: name, price and image.
        - Each product in the list can be clicked which routes to a detail page for that product.

- **(10%)** All bubble products contain a link to a detailed page for each product, where extra information is provided (e.g. **http://localhost:3000/bubbles/1**)

    - **Requirements**
        - This must be a **Server Component** that fetches data directly from the API.
        - All product images must be rendered using the Next.js <Image /> component to ensure proper optimization.
        - The product should be display:
            - Large image at the top
            - Product name as the title
            - Description
            - Price
        - A button at the bottom which says “Add to cart” which when clicked should add the product to the cart.

- **(10%)** Implement a site where you can look at different bundle products (e.g. **http://localhost:3000/bundles**). Bundle products are a combination of two or more products.

    - **Requirements**
        - This must be a **Server Component**.
        - All product images must be rendered using the Next.js <Image /> component to ensure proper optimization.
        - Display a list of bundles. Each bundle card should list the names of the products included in it and a specific "Add Bundle to Cart" button which when clicked should add the bundle to the cart.

- **(5%)** Implement a site where you can look at information on the place **Bubblify** (e.g. **http://localhost:3000/about**).

    - **Requirements**
        - This must be a **Server Component**.
        - It should display text describing the history of the company. You can make something up.

- **(10%)** Implement a site where you can look at all the products you have added to the cart (e.g. **http://localhost:3000/cart**).

- **Requirements**
  - This must be a **Client Component**.
  - Cart items should be stored in **localStorage** and retrieved when the user comes back.
  - The cart should list the selected items, their quantities, and the total price.
  - Each item in the list must have controls to increase/decrease quantity or remove the item entirely.
  - A button at the bottom which says “Proceed to checkout” which when clicked should redirect the user to the checkout phase.

- **(5%)** The navigation bar should contain a logo (*you can use any image you find suitable*) and four links. The layout should be persistent across pages.

  - Products
  - Bundles
  - About us
  - Cart

- **(35%)** A user needs to go through the following steps when buying products. This checkout flow should be implemented as multiple pages using a **React Context** to share state between the steps.

  1. Select one or more products from the product section or select previous order which can be retrieved at (**http://localhost:3500/orders/:telephone**)

  2. When a selection is finished, he gets an option to go to the checkout. The checkout should also be available in the cart

  3. In the first page of checkout (/checkout/delivery) the user needs to decide if he wants to arrange a store-pickup or have it delivered

  4. If he decides to have it delivered, the next page (/checkout/info) will ask to input the following information: **name, address, city, telephone** and **postal code**

  5. If he decides to arrange a store-pickup, the next page (/checkout/info) will ask to input only the **name** and **telephone**

  6. At the final page (/checkout/review) he gets a review of his order and if he is satisfied he can click a button to confirm

  7. When the confirm button is clicked he is redirected to a confirmation site (/checkout/success), which should tell him that all was successful. The order should be stored in the server (**http://localhost:3500/orders** [POST]). You must use a **Server Action** to handle this data submission to the external API.

8. Information about the user and the order must be persisted across these pages using a Context Provider

9. Forms should be properly validated, preventing navigation to the next step if data is missing

- **(10%)** Implement a site where you can look at your past orders (e.g. http://localhost:3000/orders).

  1. **Requirements**
     - The order list must be a **Server Component** that fetches data directly from the API.

  2. **(2%)** The landing page should provide an input for the telephone number to search for.

     - If the user types in a telephone number which has **NO** orders, the user should be shown a message that there is no order associated with that telephone number.

     - If the user types in a telephone which has associated orders, the user should be redirected to a details page for an order (e.g. /orders/5812345).

  3. **(8%)** The results page should display the order associated with that number, showing the products and total price.

- **(2.5%)** Components which receive props should have a corresponding interface setup which describes the props.

- **(2.5%)** All props passed into the component should be documented (JSDoc or standard comments).

Submission

A single compressed file (***.zip, *.rar**) should be submitted in **Canvas**. **node_modules/** and **.next/** folders should be excluded.